LINUX / SYSTEMS

SELinux Visual Guide

Security contexts, enforcing vs permissive, policies, and denial troubleshooting.

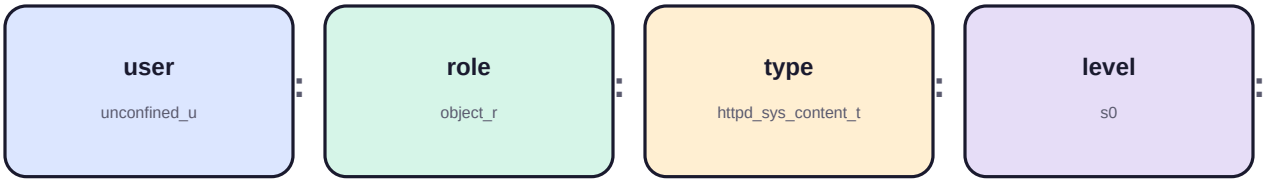
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Contexts: user:role:type:level

Every process AND every file carries a label. Policy matches labels.




```
$ ls -Z /var/www/html/index.html
unconfined_u:object_r:httpd_sys_content_t:s0 index.html
$ ps -Z | grep httpd
system_u:system_r:httpd_t:s0  1234 ?  httpd
```

The TYPE is what matters 95% of the time (type enforcement). Process type httpd_t may read file type httpd_sys_content_t — only if policy allows that pair.

STEP 2/6

Modes: Enforcing / Permissive / Disabled

Same policy, different courage.

enforcing

denies +■logs

permissive

allows +■logs

disabled

no SELinux■at all

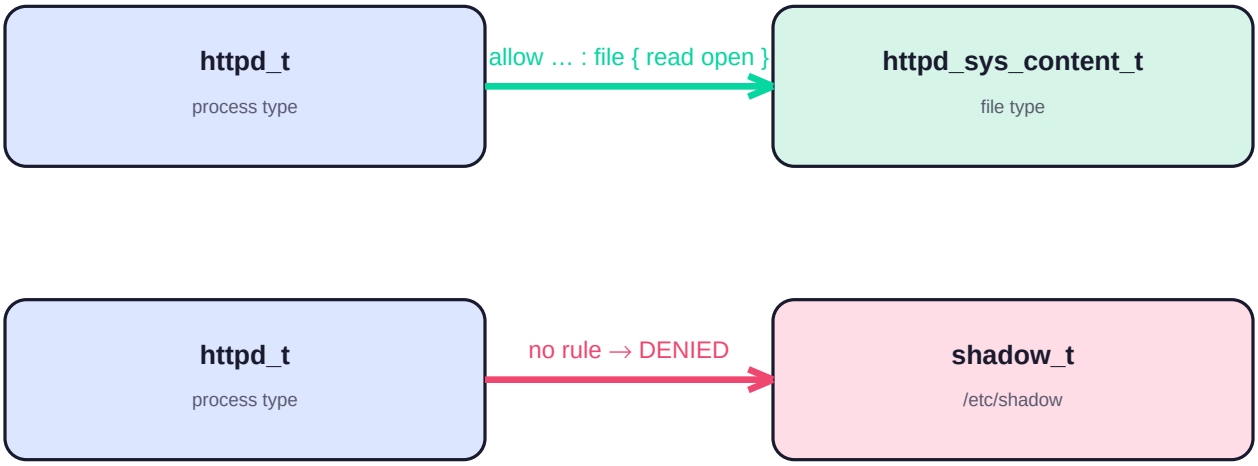
```
$ getenforce          # Enforcing?
$ sudo setenforce 0   # → permissive UNTIL REBOOT (debug!)
$ sudo setenforce 1   # → enforcing
# permanent: /etc/selinux/config → SELINUX=enforcing
```

The golden debug move: setenforce 0 — if the problem vanishes, it's SELinux. Fix the LABELS or policy, then setenforce 1. Never 'fix' it by leaving it permissive forever.

STEP 3/6

Policy: Allow Rules Between Types

Policy = which subject types may do what to which object types.




```
$ sesearch --allow -s httpd_t -t httpd_sys_content_t   # show the rule
$ sesearch --allow -s httpd_t -t shadow_t   # nothing → denied
```

STEP 4/6

Fixing Labels & Booleans

Most 'SELinux broke my app' is a wrong label, not a wrong policy.

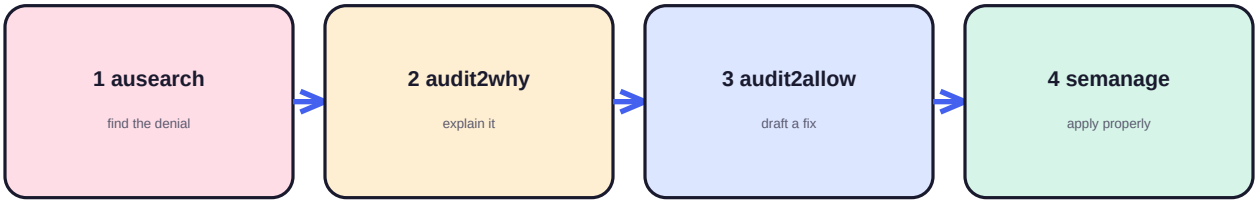
```
$ sudo restorecon -Rv /var/www/html     # relabel to policy defaults
$ sudo semanage fcontext -a -t httpd_sys_content_t '/srv/www(/.*)?'
$ sudo restorecon -Rv /srv/www          # custom path, permanent rule
$ getsebool -a | grep httpd
$ sudo setsebool -P httpd_can_network_connect on   # -P = persistent
```

- Files created in the wrong place inherit the wrong type — mv preserves, cp relabels. Classic trap.
- semanage fcontext makes custom paths survive a filesystem relabel; chcon alone does not.
- Booleans are policy feature-switches (httpd_can_network_connect, etc.) — check them before writing policy.

STEP 5/6

Troubleshooting Denials: The Drill

audit.log → why → fix. In that order.




```
$ sudo ausearch -m avc -ts recent | audit2why
type=AVC ... denied { read } ... httpd_t ... var_log_t
  Was caused by: missing type enforcement rule —
  try: restorecon, or setsebool httpd_read_logs
```

audit2allow generates policy — REVIEW it before loading. A generated 'allow everything' module is a hole you wrote yourself. Prefer relabels and booleans first.

STEP 6/6

SELinux Survival Rules

The mindset that keeps it enforcing.

- ✓ Leave it ENFORCING — permissive is a debug state, not a configuration
- ✓ setenforce 0 to test, then fix labels/policy, then setenforce 1
- ✓ restorecon before writing custom policy — 80% of denials are mislabeled files
- ✓ semanage fcontext for custom paths so relabels stick
- ✓ Check booleans (getsebool) before inventing new policy
- ✓ Read audit2allow output like a contract — never load it blind

STEP ✓/✓

Key Takeaways

SELinux

- Everything has a context user:role:type:level — type enforcement decides access.
- Enforcing denies+logs; permissive allows+logs — use permissive only to diagnose.
- Policy = allow rules between subject types and object types; no rule means denied.
- Most breakage is mislabeled files: restorecon, semanage fcontext, booleans — in that order.
- Drill: ausearch → audit2why → fix properly; never load audit2allow output blind.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev